AI Assisted Coding

Lab_Assignment_19.4

V.Sri charitha

Batch-08

2303A51626

Task 1 – JavaScript to Python (Array Processing Logic)

Scenario

A frontend application written in JavaScript processes user scores. The backend team wants the same logic implemented in Python for analytics processing.

Task

Translate a JavaScript function that:

- Takes an array of numbers
- Filters values greater than 50
- Returns the average of filtered values

Instructions

- Prompt AI to convert JavaScript array methods (filter, reduce) into equivalent Python logic.
- Ensure list comprehension or Pythonic style is used.
- Compare outputs of both implementations with sample input.
- Document differences in handling arrays between JS and Python.

Expected Output

- Equivalent Python function.
- Same output for identical inputs.
- Clean and Pythonic implementation.

Task 1 – JavaScript to Python (Array Processing Logic)

JavaScript Code

[2]
0s

```python
def average_above_50(arr):
    filtered = [x for x in arr if x > 50]
    if not filtered:
        return 0
    total_sum = sum(filtered)
    return total_sum / len(filtered)

print(average_above_50([30, 60, 80, 40, 90]))
```

```
76.66666666666667
```

Equivalent Python Code

[3]
0s

```python
def average_above_50(arr):
    filtered = [x for x in arr if x > 50]
    if not filtered:
        return 0
    return sum(filtered) / len(filtered)
print(average_above_50([30, 60, 80, 40, 90]))
```

```
76.66666666666667
```

Observation:

- JavaScript uses array methods like filter() and reduce(), while Python uses list comprehension and built-in functions like sum().
- Python code is more concise and readable compared to JavaScript.
- Both implementations produce the same output for identical inputs.
- Python handles empty lists easily using if not list, while JavaScript uses length.
- The logic remains the same, only the syntax differs

Task 2 – Python OOP to C++ Class Translation

Scenario

A data modeling module is written in Python using OOP. The system is being migrated to a high-performance C++ environment.

Task

Translate a Python class that:

• Contains private attributes

• Uses getter/setter methods

• Includes a method that calculates a derived value

Instructions

• Convert the Python class into a C++ class.

• Ensure:

o Proper access modifiers

o Constructor handling

o Correct data types

• Compile and test the C++ version.

• Compare how encapsulation differs between Python and C++.

Expected Output

• Fully working C++ class equivalent to Python version.

• Proper use of access specifiers (private, public).

Task 2 – Python OOP to C++ Class Translation

Python Class

[4]
0s

```python
class Product:
    def __init__(self, price, quantity):
        self.__price = price
        self.__quantity = quantity

    def get_price(self):
        return self.__price

    def set_price(self, price):
        self.__price = price

    def total_value(self):
        return self.__price * self.__quantity
```

Equivalent C++ Class

[7]
0s

```shell
%%shell
g++ -o product_main product_code.cpp
./product_main
```

```
301.5
```

Observation:

- Python uses name mangling (__var) for private variables, while C++ uses strict access modifiers (private).
- C++ requires explicit data types, whereas Python is dynamically typed.
- Constructors in C++ are more structured compared to Python's __init_.
- Encapsulation is strongly enforced in C++, but more flexible in Python.
- Both classes achieve the same functionality with different syntax rules.

Task 3 – REST API Call Translation: Python to JavaScript

Scenario

A backend developer wrote an API request using Python requests. The

frontend team needs the same functionality in JavaScript (Fetch API).

Task

Translate Python API request code into JavaScript.

Instructions

• Convert:

o GET request logic

o JSON parsing

o Error handling (try-except → try-catch)

• Ensure proper async/await usage in JavaScript.

• Compare differences in error handling mechanisms.

Expected Output

• Equivalent JavaScript code using Fetch API.

• Proper async handling.

• Similar output structure.

Python Code

```python
import requests

try:
    response = requests.get("https://jsonplaceholder.typicode.com/posts")
    data = response.json()
    print(data)
except Exception as e:
    print(e)
```

```
[{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenc
```

JavaScript Equivalent

```javascript
%%javascript
async function fetchData() {
    try {
        const response = await fetch("https://jsonplaceholder.typicode.com/posts");
        const data = await response.json();
        console.log(data);
    } catch (error) {
        console.error(error);
    }
}

fetchData();
```

Observation:

- Python uses the **requests library**, while JavaScript uses the **Fetch API**.
- JavaScript handles API calls **asynchronously using async/await**, whereas Python is usually synchronous.
- Error handling is done using **try-except in Python** and **try-catch in JavaScript**.
- Both approaches successfully retrieve and process **JSON data**.
- JavaScript requires more handling for asynchronous execution.

Task 4 – SQL Aggregation to MongoDB Query Translation

Scenario

A company is migrating from relational databases (SQL) to MongoDB (NoSQL).

Task

Translate an SQL aggregation query into a MongoDB aggregation pipeline.

Example Logic:

• Group records by category

• Calculate average price

• Filter groups where average price > threshold

Instructions

• Provide SQL query to AI.

• Generate equivalent MongoDB query.

• Explain differences between relational and document-based querying.

• Validate results conceptually.

Expected Output

• Working MongoDB aggregation query.

• Clear understanding of grouping and aggregation differences

Task 4 – SQL Aggregation to MongoDB Query Translation

SQL Query

[28]
0s

```
import sqlite3

conn = sqlite3.connect("example.db")
cursor = conn.cursor()

# Create the 'products' table if it doesn't exist
cursor.execute("""
CREATE TABLE IF NOT EXISTS products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    category TEXT NOT NULL,
    price REAL NOT NULL
);
""")

# Insert sample data if the table is empty
cursor.execute("SELECT COUNT(*) FROM products;")
if cursor.fetchone()[0] == 0:
    sample_data = [
        ('Electronics', 150),
        ('Electronics', 200),
        ('Books', 50),
        ('Books', 120),
        ('Electronics', 300),
```

```
[28]
0s
            ('Electronics', 300),
            ('Books', 80),
            ('Clothing', 75),
            ('Clothing', 110)
        ]
        cursor.executemany("INSERT INTO products (category, price) VALUES (?, ?);", sample_data)
        conn.commit()
        print("Inserted sample data into 'products' table.")
    else:
        print("Table 'products' already contains data. Skipping sample data insertion.")

    query = """
    SELECT category, AVG(price) AS avg_price
    FROM products
    GROUP BY category
    HAVING AVG(price) > 100;
    """

    cursor.execute(query)

    print("\nSQL Aggregation Results:")
    for row in cursor.fetchall():
        print(row)

    conn.close()

Inserted sample data into 'products' table.

SQL Aggregation Results:
```

```
[28]
0s
    """

    cursor.execute(query)

    print("\nSQL Aggregation Results:")
    for row in cursor.fetchall():
        print(row)

    conn.close()

Inserted sample data into 'products' table.

SQL Aggregation Results:
('Electronics', 216.66666666666666)

[23]
7s
!pip install pymongo

Collecting pymongo
  Downloading pymongo-4.16.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_6
Collecting dnspython<3.0.0,>=2.6.1 (from pymongo)
  Downloading dnspython-2.8.0-py3-none-any.whl.metadata (5.7 kB)
Downloading pymongo-4.16.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.7/1.7 MB 61.8 MB/s eta 0:00:00
Downloading dnspython-2.8.0-py3-none-any.whl (331 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 331.1/331.1 kB 28.4 MB/s eta 0:00:00
Installing collected packages: dnspython, pymongo
Successfully installed dnspython-2.8.0 pymongo-4.16.0
```

Observation:

- SQL uses **relational tables**, while MongoDB uses **document-based collections**.
- SQL performs aggregation using GROUP BY and HAVING, whereas MongoDB uses $group and $match.

- MongoDB stores grouped results in _id, unlike SQL which keeps column names.
- The aggregation pipeline in MongoDB works in **stages**, unlike SQL's single query format.
- Both produce the **same result but follow different data models and syntax**.

Task 5 – Real-Time Application: Algorithm Translation Across Three Languages

Scenario

A tech company maintains codebases in multiple languages (Python, Java, and Go). A common algorithm must be implemented consistently across systems.

Task

Choose an algorithm (e.g., binary search, palindrome check, prime number checker) and translate it across:

• Python

• Java

• One additional language (Go / C# / JavaScript)

Instructions

• Ensure logic remains identical across languages.

• Handle:

o Data types

o Loop differences

o Input/output handling

• Test each version with same input values.

• Briefly document translation challenges.

Expected Output

• Three equivalent working implementations.

- Clear logical consistency.
- Understanding of syntactic vs structural differences.

Task 5 – Real-Time Application: Algorithm Translation Across Three Languages

Python

[15]
0s

```python
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

print(binary_search([1,2,3,4,5], 4))
```

```
3
```

Java

[20]
0s

```
%%writefile BinarySearch.java
class BinarySearch {
    static int search(int[] arr, int target) {
        int low = 0, high = arr.length - 1;
```

[20]
0s

```
%%writefile BinarySearch.java
class BinarySearch {
    static int search(int[] arr, int target) {
        int low = 0, high = arr.length - 1;
        while (low <= high) {
            int mid = (low + high) / 2;
            if (arr[mid] == target)
                return mid;
            else if (arr[mid] < target)
                low = mid + 1;
            else
                high = mid - 1;
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] arr = {1,2,3,4,5};
        System.out.println(search(arr, 4));
    }
}
```

```
Writing BinarySearch.java
```

[21]
1s

```
%%shell
javac BinarySearch.java
```

[22]
0s

```
%%shell
java BinarySearch
```

```
3
```

Java Script

[19]
0s

```javascript
%%javascript
function binarySearch(arr, target) {
    let low = 0, high = arr.length - 1;
    while (low <= high) {
        let mid = Math.floor((low + high) / 2);
        if (arr[mid] === target) return mid;
        else if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

console.log(binarySearch([1,2,3,4,5], 4));
```

...

Observation:

- The algorithm logic remains identical across all languages, ensuring consistent results.
- Differences arise in syntax, data types, and structure.
- Java requires a class structure, while Python and JavaScript are more flexible.
- Loop constructs and condition handling vary slightly between languages.
- Translation highlights the difference between language syntax and core algorithm logic.